

Práctica 2 - HTTP

1. ¿Cuál es la función de la capa de aplicación?

- Proveer servicios de comunicación a los usuarios (“Capa 8”) y a las aplicaciones.
- Incluir las aplicaciones mismas dentro de la capa, no solo los protocolos.
- Soportar comunicación Machine-to-Machine (M2M): incluso cuando no hay usuarios humanos.
- Interfaz con el usuario (User Interface - UI) u otras aplicaciones/servicios.
- Clasificar las aplicaciones que usan la red como parte de esta capa.
- Definir protocolos de aplicación (HTTP, FTP, SMTP, etc.) que implementan las aplicaciones.
- Integrar aplicaciones que no son de red con servicios de red para que puedan acceder a ella

2. Si dos procesos deben comunicarse:

a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

b. Y si están en la misma máquina, ¿qué alternativas existen?

Ubicación de los procesos	Cómo se comunican
Diferentes máquinas	- Identificador único = Dirección del host (IP) + Puerto. - Comunicación mediante sockets (BSD, Winsock, TLI/XTI). - Capa de transporte (TCP/UDP) provee multiplexación para dirigir mensajes al proceso correcto.
Misma máquina	- No se necesita dirección de host. - - Uso de IPC (Inter-Process Communication): • Pipes y FIFOs • Memoria compartida • Semáforos/Mutex para sincronización • Colas de mensajes • Sockets locales (Unix Domain Sockets)

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la “vida cotidiana” y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

Es un modelo de carga compartida:

- El **cliente** se encarga de la interfaz y parte del procesamiento.
- El **servidor** se encarga del resto del procesamiento y atiende solicitudes de múltiples clientes.
- El servidor espera conexiones de forma pasiva y responde a las peticiones de los clientes.
- Puede haber variantes en varios tiers (2-tier, 3-tier, multi-tier).

Ejemplos

- Vida cotidiana: Un restaurante → el cliente pide (cliente), el mozo lleva el pedido a la cocina (servidor), la cocina prepara y entrega la comida (respuesta).
- Sistema informático: Un navegador web que solicita una página a un servidor web mediante el protocolo HTTP.

Otros modelos de comunicación

- Además del cliente/servidor, tu PDF menciona:
- Mainframe (Dumb Client): el cliente solo muestra datos, todo el procesamiento está en el servidor.
- Peer-to-Peer (P2P): todos los nodos son pares, pueden actuar como clientes y servidores al mismo tiempo.
- Modelo Híbrido: combina cliente/servidor y P2P (por ejemplo, Napster o Skype)

4. Describa la funcionalidad de la entidad genérica “Agente de usuario” o “User agent”

```
GET /index2.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
```

Funcionalidad de la entidad genérica User Agent

- Es la aplicación cliente que actúa en nombre del usuario para interactuar con el servidor.
- En el contexto de HTTP, el *User-Agent* es un header que identifica el software cliente que realiza el request (por ejemplo, un navegador, una app, un script).
- Permite al servidor adaptar la respuesta según el tipo de cliente. Ejemplo: un sitio puede devolver una versión móvil si detecta que el User-Agent es un navegador de teléfono.
- Puede incluir información como:
 - Nombre de la aplicación (ej. Chrome, Firefox, cURL)
 - Versión
 - Sistema operativo o plataforma
- Es fundamental para **estadísticas de uso, debugging** y para algunas decisiones de compatibilidad del lado del servidor

5. ¿Qué son y en qué se diferencian HTML y HTTP?

🚩 Comparación: HTML vs HTTP		
Característica	HTML (HyperText Markup Language)	HTTP (HyperText Transfer Protocol)
Qué es	Lenguaje de marcado.	Protocolo de comunicación de la capa de aplicación.
Función	Define la estructura y formato de las páginas web (texto, imágenes, enlaces, formularios).	Define cómo cliente y servidor intercambian mensajes (request/response).
Dónde actúa	Lado del cliente (el navegador interpreta el HTML y lo muestra en pantalla).	En la comunicación cliente-servidor , sobre TCP (puerto 80 o 443 si es HTTPS).
Ejemplo	Hola mundo → se muestra un título en la página.	GET /index.html HTTP/1.1 → el cliente pide un recurso al servidor.
Tipo de estándar	Estándar de contenido y presentación (W3C).	Estándar de protocolo de transporte (IETF – RFC 1945, RFC 2616).
Qué transporta	N/A (es el propio contenido).	Transporta documentos HTML y otros objetos (CSS, imágenes, JSON, etc.).

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado “Formato de mensaje HTTP”, Kurose).

a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información? ¿Para qué sirven las cabeceras?

🔧 Comparación: Request vs Response en HTTP		
Parte	Request (Petición)	Response (Respuesta)
Primera línea	Request Line → <Método> <URI> <Versión>Ej: GET / index.html HTTP/1.1	Status Line → <Versión> <Código> <Frase>Ej: HTTP/ 1.1 200 OK
Cabeceras (Headers)	Información del cliente:- Host → servidor solicitado- User-Agent → navegador/app- Accept → formatos aceptados	Información del servidor:- Content-Type → tipo de recurso- Content-Length → tamaño- Set-Cookie, Date, etc.
Línea en blanco	Separa cabeceras del cuerpo	Separa cabeceras del cuerpo
Cuerpo (Body)	Opcional → datos enviados (ej: formulario en POST)	Opcional → recurso solicitado (HTML, imagen, JSON...)

🔧 Ejemplo de Request

http

Copy

```
GET /index.html HTTP/1.1
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
Accept: text/html
```

🔧 Ejemplo de Response

http

Copy

```
HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:28:51 GMT
Server: Apache/2.2.4 (Ubuntu)
Content-Length: 31
Content-Type: text/html
Connection: close

<html>
<h1>HOLA</h1>
</html>
```

🔧 Ejemplo de Response sin Body

http

Copy

```
HTTP/1.1 204 No Content
Date: Mon, 21 Apr 2008 00:28:51 GMT
Server: Apache/2.2.4 (Ubuntu)
Connection: close
```

b. ¿Cuál es su formato?

🔧 Formato de las Cabeceras HTTP		
Elemento	Formato	Ejemplo
Estructura básica	Nombre-de-Cabecera: Valor	Content-Type: text/html
Mayúsculas/minúsculas	No importa (case-insensitive).	host: = Host:
Separador	Dos puntos : seguidos de un espacio antes del valor.	User-Agent: Mozilla/5.0
Múltiples valores	Pueden separarse con comas.	Accept: text/html, application/ json
Fin de cabeceras	Línea en blanco antes del cuerpo (body).	(vacío)

c. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como `www.misitio.com` para obtener el recurso `/index.html`. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.

```
GET /index.html HTTP/1.1
Host: www.misitio.com
User-Agent: curl/7.74.0
Accept: text/html
```

Notas

- Se puede usar `Accept: */*` para aceptar cualquier tipo de contenido.
- La línea en blanco al final es obligatoria (marca el fin de las cabeceras).
- No hay body, porque es un GET.

7. Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s). Ejemplos de cada parámetro:

📌 Parámetros de curl		
Parámetro	Función	Ejemplo de uso
-I	Realiza un HEAD request → solo obtiene las cabeceras de respuesta, no el cuerpo del recurso. Sirve para verificar si un recurso existe o para leer metadatos (Content-Type, Content-Length, etc.).	<code>curl -I https://www.ejemplo.com</code>
-H	Permite agregar o modificar headers en el request. Muy útil para enviar encabezados personalizados (ej. tokens, Accept, User-Agent).	<code>curl -H "Accept: application/json" https://api.ejemplo.com</code>
-X	Especifica el método HTTP a usar (por defecto es GET). Se usa para hacer POST, PUT, DELETE, etc.	<code>curl -X POST https://www.ejemplo.com</code>
-s	Modo silencioso (silent) → no muestra la barra de progreso ni mensajes de error. Devuelve solo la salida del recurso (útil para scripts).	<code>curl -s https://www.ejemplo.com</code>

```
redes@debian:~$ curl -I www.wikipedia.com
HTTP/1.1 301 Moved Permanently
Server: nginx/1.22.1
Date: Mon, 15 Sep 2025 14:44:49 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://www.wikipedia.com/
```

```
redes@debian:~$ curl -H "User-Agent: MiVM/1.0" -H "Accept: application/json" https://httpbin.org/headers
{
  "headers": {
    "Accept": "application/json",
    "Host": "httpbin.org",
    "User-Agent": "MiVM/1.0",
    "X-Amzn-Trace-Id": "Root=1-68c828f9-1be3cbf730ad66587eee13ff"
  }
}
```

```
redes@debian:~$ curl -X POST -d "nombre=NicoPedro" https://httpbin.org/post
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "nombre": "NicoPedro"
  },
  "headers": {
    "Accept": "*/*",
    "Content-Length": "16",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.74.0",
    "X-Amzn-Trace-Id": "Root=1-68c82a54-3e9c49587b9a2a5853b3fba4"
  },
  "json": null,
  "origin": "190.227.35.46",
  "url": "https://httpbin.org/post"
}
```

```
redes@debian:~$ curl -s https://postman-echo.com/get?foo=bar
{
  "args": {
    "foo": "bar"
  },
  "headers": {
    "host": "postman-echo.com",
    "x-request-start": "t1757948974.151",
    "connection": "close",
    "x-forwarded-proto": "https",
    "x-forwarded-port": "443",
    "x-amzn-trace-id": "Root=1-68c82c2e-13fa492856696cde40fbd434",
    "user-agent": "curl/7.74.0",
    "accept": "*/*"
  },
  "url": "https://postman-echo.com/get?foo=bar"
}
```

8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (>) del comando curl a un archivo con extensión html y abrirlo con un navegador.

El comando `curl www.redes.unlp.edu.ar` realizó un único requerimiento HTTP al servidor y obtuvo como respuesta el documento HTML principal del sitio. No descargó automáticamente los recursos adicionales (hojas de estilo, imágenes, scripts).

Si se redirige la salida a un archivo (`curl www.redes.unlp.edu.ar > index.html`) y se abre con un navegador, este intentará cargar los recursos referenciados en el HTML desde sus respectivas ubicaciones en la web.

b. ¿Cómo funcionan los atributos href de los tags link e img en html?

Los atributos href (en etiquetas <link> o <a>) y src (en etiquetas , <script>, etc.) contienen URLs absolutas o relativas que indican la ubicación de otros recursos. El navegador, al interpretar el HTML, genera nuevos requerimientos HTTP para cada recurso referenciado con el fin de renderizar la página de manera completa.

c. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?

Para mostrar la página tal como se vería en un navegador, no alcanza con un único requerimiento, ya que el HTML hace referencia a otros objetos (CSS, imágenes, scripts).

d. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente

En el caso de una página que contiene dos hojas de estilo (CSS), dos scripts (JavaScript) y tres imágenes, serían necesarios como mínimo:

- 1 requerimiento para el documento HTML principal
- 2 para los archivos CSS
- 2 para los archivos JavaScript
- 3 para las imágenes

Total: 8 requerimientos HTTP.

Diferencia entre navegador y curl

- **Navegador:** interpreta el HTML, sigue automáticamente los enlaces a recursos (href, src), realiza múltiples requerimientos en paralelo y muestra el resultado completo (texto, imágenes, estilos). Además, utiliza mecanismos como cache, keep-alive y manejo de redirecciones para optimizar la carga.
- **curl:** realiza solamente el requerimiento solicitado y muestra el contenido tal cual lo recibe (generalmente el HTML). No interpreta el documento ni solicita los recursos adicionales, a menos que se invoque explícitamente para cada uno de ellos.

9. Ejecute a continuación los siguientes comandos:

- `curl -v -s www.redes.unlp.edu.ar > /dev/null`

```
redes@debian:~$ curl -v -s www.redes.unlp.edu.ar > /dev/null
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Mon, 15 Sep 2025 16:38:54 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
{ [4898 bytes data]
* Connection #0 to host www.redes.unlp.edu.ar left intact
```

- `curl -I -v -s www.redes.unlp.edu.ar`

```
redes@debian:~$ curl -I -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> HEAD / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Date: Mon, 15 Sep 2025 16:39:20 GMT
Date: Mon, 15 Sep 2025 16:39:20 GMT
< Server: Apache/2.4.56 (Unix)
Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
Accept-Ranges: bytes
< Content-Length: 4898
Content-Length: 4898
< Content-Type: text/html
Content-Type: text/html
<
* Connection #0 to host www.redes.unlp.edu.ar left intact
```

a. ¿Qué diferencias nota entre cada uno?

Aspecto	<code>curl -v -s www.redes.unlp.edu.ar > /dev/null</code>	<code>curl -I -v -s www.redes.unlp.edu.ar</code>
Método usado	GET (trae HTML)	HEAD (solo cabeceras)
Cuerpo de la respuesta	El servidor envía el body (4.898 bytes), pero se descarta con <code>> /dev/null</code> .	El servidor no envía body (HEAD nunca tiene contenido).
Velocidad / transferencia	Descarga el recurso completo aunque no se muestre.	Más rápido: solo transmite cabeceras.
Cabeceras mostradas	Se ven cabeceras de request y response, pero además aparece el mensaje <code>{ [4898 bytes data] }</code> indicando que había un body.	Solo se ven las cabeceras; no hay datos ni indicación de tamaño descargado.
En resumen: el primer comando realiza un GET normal y descarta el contenido, mientras que el segundo hace un HEAD, evitando que el servidor envíe el contenido en primer lugar.		

b. ¿Qué ocurre si en el primer comando se quita la redirección a /dev/null? ¿Por qué no es necesaria en el segundo comando?

- En el primer comando es necesaria para no imprimir el contenido HTML en la terminal.
- En el segundo comando no es necesaria, porque el método HEAD no devuelve contenido.

c. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?

Request enviado por curl:

1. GET / HTTP/1.1 (o HEAD / HTTP/1.1 en el segundo caso)
2. Host: www.redes.unlp.edu.ar
3. User-Agent: curl/7.74.0
4. Accept: */*

Total: 4 líneas (1 línea de request + 3 cabeceras)

Response devuelta por el servidor:

1. HTTP/1.1 200 OK
2. Date:
3. Server:
4. Last-Modified:
5. ETag:
6. Accept-Ranges:
7. Content-Length:
8. Content-Type:

Total: 8 cabeceras en la respuesta (además de la línea de estado).

*Nota: por qué aparecen dos veces las cabeceras del response con **curl -I -v -s www.redes.unlp.edu.ar** ?*

Las cabeceras duplicadas que aparecen con `curl -I -v` no provienen del servidor; son el resultado del modo verbose, que muestra tanto la salida de depuración (<) como la respuesta normal de -I.
Si se ejecuta sin -v, las cabeceras aparecen solo una vez.

10. ¿Qué indica la cabecera Date?

La cabecera Date indica la fecha y hora en que el servidor generó la respuesta.

Propósito:

- Permite a clientes y proxies saber cuándo fue creada la respuesta.
- Sirve como referencia para mecanismos de caché, expiración y validación de contenido.

11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

Comparación: Detección de fin de objeto en HTTP

Versión	Cómo sabe el cliente que terminó de recibir el objeto	Comentarios
HTTP/1.0	El servidor cierra la conexión TCP cuando termina de enviar el objeto.	Ineficiente: para cada objeto (HTML, CSS, imágenes) se debe abrir una nueva conexión.
HTTP/1.1	- Usa la cabecera Content-Length para indicar la longitud del objeto.- Si no se conoce el tamaño de antemano, utiliza Transfer-Encoding: chunked y envía el objeto en fragmentos hasta un chunk de longitud cero.	Permite conexiones persistentes (Keep-Alive) para enviar múltiples objetos en la misma conexión.

Resumen

- HTTP/1.0: el fin de la conexión = fin del objeto.
- HTTP/1.1: el tamaño se indica explícitamente (Content-Length o chunked), manteniendo la conexión abierta.

12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

Código	Descripción
200	OK – La solicitud ha tenido éxito. La información devuelta depende del método utilizado (por ejemplo, con GET se devuelve el recurso solicitado).
201	Created – La solicitud ha sido cumplida y ha resultado en la creación de un nuevo recurso.
204	No Content – El servidor ha cumplido la solicitud pero no devuelve contenido en el cuerpo de la respuesta.
206	Partial Content – El servidor devuelve solo una parte del recurso, como respuesta a una petición con cabecera Range.
301	Moved Permanently – El recurso solicitado se ha movido permanentemente a una nueva URL. El cliente debe usar la nueva dirección en el futuro.
302	Found – El recurso solicitado reside temporalmente en otra URL. El cliente debe usar la dirección original en futuras solicitudes.
304	Not Modified – Indica que el recurso no ha sido modificado desde la última vez que fue solicitado. Se utiliza con mecanismos de caché.
401	Unauthorized – La solicitud requiere autenticación. El cliente debe enviar credenciales válidas.
403	Forbidden – El servidor entendió la solicitud, pero se niega a cumplirla (no es un problema de autenticación).
404	Not Found – El servidor no encontró el recurso solicitado.
409	Conflict – La solicitud no pudo completarse debido a un conflicto con el estado actual del recurso.
500	Internal Server Error – Error interno del servidor. El servidor encontró una condición inesperada que le impidió cumplir la solicitud.

Categoría	Significado	Ejemplos comunes
2XX – Éxito	El request fue procesado correctamente.	200 OK: recurso devuelto con éxito. 201 Created: recurso creado (POST). 204 No Content: éxito sin contenido en el body.
3XX – Redirección	Se requiere acción adicional para completar el request (generalmente seguir otra URL).	301 Moved Permanently: URL cambiada de forma permanente. 302 Found: redirección temporal. 304 Not Modified: usar copia en caché.
4XX – Error del cliente	El problema está en el request del cliente.	400 Bad Request: sintaxis incorrecta. 401 Unauthorized: requiere autenticación. 403 Forbidden: acceso no permitido. 404 Not Found: recurso no encontrado.
5XX – Error del servidor	El servidor no pudo procesar el request correctamente.	500 Internal Server Error: error genérico. 502 Bad Gateway: respuesta inválida de otro servidor. 503 Service Unavailable: servidor no disponible. 504 Gateway Timeout: tiempo de espera agotado.

13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

```
redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Mon, 15 Sep 2025 17:09:58 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 4898
Content-Type: text/html
```

Diferencia -I vs -X HEAD (con curl)

Opción	Qué hace	Cuándo usar
-I	Hace un HEAD y muestra solo las cabeceras (no el body).	Es el atajo más usado para obtener únicamente headers.
-X HEAD -i	Fuerza el método HEAD de forma explícita y muestra las cabeceras porque usamos -i.	Útil si querés combinar con otras opciones (-H, -v) y controlar el request manualmente.

Resumen:

- -I = atajo rápido.
- -X HEAD = forma explícita, más flexible.

a) ¿Qué información brinda la primera línea de la respuesta?

La línea **HTTP/1.1 200 OK** indica:

- Protocolo y versión: HTTP/1.1
- Código de estado: 200
- Mensaje: OK

Por lo tanto, el servidor procesó el requerimiento correctamente.

b) ¿Cuántos encabezados muestra la respuesta?

La respuesta incluye 7 encabezados:

1. Date
2. Server
3. Last-Modified
4. ETag
5. Accept-Ranges
6. Content-Length
7. Content-Type

c) ¿Qué servidor web está sirviendo la página?

El encabezado **Server: Apache/2.4.56 (Unix)** indica que el servidor es Apache versión 2.4.56 ejecutándose sobre Unix.

d) ¿El acceso a la página solicitada fue exitoso o no?

Sí. El código de estado **200 OK** indica que el acceso fue exitoso y el recurso se devolvió correctamente.

e) ¿Cuándo fue la última vez que se modificó la página?

El encabezado **Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT** indica que la última modificación del recurso fue el 19 de marzo de 2023 a las 19:04:46 GMT.

f. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?

```
redes@debian:~$ curl -i -H "If-Modified-Since: Mon, 20 Mar 2023 00:00:00 GMT" www.redes.unlp.edu.ar
HTTP/1.1 304 Not Modified
Date: Mon, 15 Sep 2025 17:18:28 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
```

Explicación:

- -i → muestra cabeceras + contenido (si lo hay).
- -H → agrega un header personalizado.
- If-Modified-Since → el servidor solo envía el recurso si fue modificado después de la fecha indicada.

Como en la cabecera Last-Modified vimos que la página fue modificada el 19 de marzo de 2023, al pedirla con una fecha posterior (20 de marzo 2023):

- El servidor debería responder con:

```
HTTP/1.1 304 Not Modified
```

- Sin cuerpo, ya que el recurso no fue modificado desde la fecha indicada.

14. Utilizando curl, acceda al sitio www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.

1. Primer acceso:

```
redes@debian:~$ curl www.redes.unlp.edu.ar/restringido/index.php
<h1>Acceso restringido</h1>

<p>Para acceder al contenido es necesario autenticarse. Para obtener los datos de acceso seguir las instrucciones detalladas en www.redes.unlp.edu.ar/obtener-usuario.php</p>
```

El servidor respondió con un mensaje indicando que era necesario autenticarse y que se debía consultar obtener-usuario.php para obtener las credenciales.

2. Obtención de credenciales:

```
redes@debian:~$ curl -i -H "Usuario-Redes: obtener" www.redes.unlp.edu.ar/obtener-usuario.php
HTTP/1.1 200 OK
Date: Mon, 15 Sep 2025 17:24:41 GMT
Server: Apache/2.4.56 (Unix)
X-Powered-By: PHP/7.4.33
Content-Length: 286
Content-Type: text/html; charset=UTF-8

<p>Bien hecho! Los datos para ingresar son:

    Usuario: redes

    Contraseña: RYC

    Ahora vuelva a acceder a la página inicial con los datos anteriores.

    PISTA: Investigue el uso del encabezado Authorization para el método Basic.
    El comando base64 puede ser de ayuda!</p>
```

La respuesta devolvió el usuario y contraseña para autenticarse.

3. Acceso con autenticación básica:

```
redes@debian:~$ curl -i -u redes:RYC www.redes.unlp.edu.ar/restringido/index.php
HTTP/1.1 302 Found
Date: Mon, 15 Sep 2025 17:25:04 GMT
Server: Apache/2.4.56 (Unix)
X-Powered-By: PHP/7.4.33
Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
Content-Length: 230
Content-Type: text/html; charset=UTF-8

<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>
```

El servidor respondió con 302 Found y la cabecera Location apuntando a the-end.php.

4. Seguir redirección automáticamente:

```
redes@debian:~$ curl -L -i -u redes:RYC www.redes.unlp.edu.ar/restringido/index.php
HTTP/1.1 302 Found
Date: Mon, 15 Sep 2025 17:26:52 GMT
Server: Apache/2.4.56 (Unix)
X-Powered-By: PHP/7.4.33
Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
Content-Length: 230
Content-Type: text/html; charset=UTF-8

HTTP/1.1 200 OK
Date: Mon, 15 Sep 2025 17:26:52 GMT
Server: Apache/2.4.56 (Unix)
X-Powered-By: PHP/7.4.33
Content-Length: 159
Content-Type: text/html; charset=UTF-8

¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2025-09-15 17:26:52
Verificación: 94b0a4dbb318eb21062ff679bb9ac2897e88f27ddb6672533654d18520b75123re
```

Explicación:

- `-L` → sigue redirecciones automáticamente (302, 301, etc.).
- `-i` → muestra cabeceras + cuerpo.
- `-u redes:RYC` → envía las credenciales en todos los requests de la cadena.

Curl siguió la redirección y mostró la respuesta final con código 200 OK.

Conclusión: se practicó el uso de headers personalizados (`-H`), autenticación HTTP básica (`-u`), análisis de cabeceras (`WWW-Authenticate`, `Location`, `Content-Type`) y manejo de redirecciones (`-L`). El ejercicio finalizó con la obtención de la cadena de verificación solicitada.

15. Utilizando la VM, realice las siguientes pruebas:

a. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).

```
redes@debian:~$ curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

b. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?

```
redes@debian:~$ telnet www.redes.unlp.edu.ar 80
Trying 172.28.0.59...
Connected to www.redes.unlp.edu.ar.
Escape character is '^]'.
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

HTTP/1.1 200 OK
Date: Mon, 15 Sep 2025 17:45:31 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "768-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Connection: close
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Protocolo HTTP: versiones</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="">
<meta name="author" content="">
<!-- Le styles -->
<link href="../../bootstrap/css/bootstrap.css" rel="stylesheet">
<link href="../../css/style.css" rel="stylesheet">
<link href="../../bootstrap/css/bootstrap-responsive.css" rel="stylesheet">
<!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
<!--[if lt IE 9]>
<script src="../../bootstrap/js/html5shiv.js"></script>
<![endif]-->
</head>
<body>

<div id="wrap">

<div class="navbar navbar-inverse navbar-fixed-top">
<div class="navbar-inner">
<div class="container">
<a class="brand" href="../../index.html"><i class="icon-home icon-whi
<a class="brand" href="https://catedras.info.unlp.edu.ar" target="_bl
<a class="brand" href="http://www.info.unlp.edu.ar" target="_blank">F
<a class="brand" href="http://www.unlp.edu.ar" target="_blank">UNLP</
</div>
</div>
</div>

<div class="container">
<h1>Ejemplo del protocolo HTTP 1.1</h1>
<p>
Esta página se visualiza utilizando HTTP 1.1. Utilizando el capt
</p>
</p>
<h2>Imagen de ejemplo</h2>

</div>

</div>
<div id="footer">
<div class="container">
<p class="muted credit">Redes y Comunicaciones</p>
</div>
</div>
</body>
</html>
Connection closed by foreign host.
```

Análisis del resultado

- La conexión se estableció correctamente con el servidor (Connected to www.redes.unlp.edu.ar).
- Se envió la petición utilizando HTTP/1.0, incluyendo las cabeceras correspondientes y el doble salto de línea final requerido para indicar el fin del request.
- El servidor respondió con:
- Línea de estado: HTTP/1.1 200 OK, indicando que la solicitud fue procesada con éxito.
- Cabeceras de respuesta: Date, Server, Content-Length, Connection: close, Content-Type, entre otras.
- Cuerpo del mensaje: el contenido HTML completo de la página solicitada.

Observación: la presencia de la cabecera

Connection: close implica que el servidor cierra la conexión una vez finalizada la transmisión de la respuesta.

Por este motivo, al finalizar la descarga del HTML, la sesión de telnet queda cerrada y no es posible enviar nuevas solicitudes sin establecer una nueva conexión.

c. Repita el proceso anterior, pero copiando la salida del recurso /extras/prueba-http-1-1.txt. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.

Análisis del resultado con HTTP/1.1

- Se estableció nuevamente la conexión con el servidor mediante telnet.
- Se envió la petición utilizando HTTP/1.1, incluyendo las cabeceras correspondientes (Host, User-Agent, Accept) y el doble salto de línea final.
- El servidor respondió con:
 - Línea de estado: HTTP/1.1 200 OK, indicando una respuesta exitosa.
 - Cabeceras de respuesta: similares al caso anterior, pero sin la cabecera Connection: close.
 - Cuerpo del mensaje: el contenido HTML completo del recurso solicitado.

Observación: la ausencia de Connection: close indica que, bajo HTTP/1.1, la conexión permanece abierta de manera predeterminada (persistent connection). Esto permite enviar múltiples peticiones dentro de la misma sesión telnet sin necesidad de reabrir la conexión para cada request, a diferencia de lo que ocurre en HTTP/1.0.

16. En base a lo obtenido en el ejercicio anterior, responda:

Pregunta	Respuesta
a) ¿Qué está haciendo al ejecutar el comando telnet?	Establece manualmente una conexión TCP con el servidor en el puerto 80 y permite enviar peticiones HTTP de forma directa, observando el intercambio de mensajes sin intermediarios.
b) ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?	Se utilizó el método GET . Se solicitaron los recursos /extras/prueba-http-1-0.txt (HTTP/1.0) y /extras/prueba-http-1-1.txt (HTTP/1.1).
c) ¿Qué diferencias notó entre los dos casos? ¿Por qué?	- HTTP/1.0: el servidor responde y cierra la conexión (cabecera Connection: close).- HTTP/1.1: la conexión permanece abierta de forma predeterminada, permitiendo múltiples requests sin reconectar. 🚩 Diferencia: HTTP/1.1 soporta conexiones persistentes por defecto, a diferencia de HTTP/1.0.
d) ¿Cuál es más eficiente? ¿Puede traer problemas?	HTTP/1.1 es más eficiente porque reutiliza la misma conexión para varios recursos, reduciendo latencia y costo de apertura de conexiones. 🚩 Posible problema: si las conexiones permanecen abiertas demasiado tiempo, pueden ocupar recursos del servidor y degradar el rendimiento si no se gestionan los timeouts.

17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET .

🚩 Comparación de GET vs POST en HTTP		
Aspecto	GET	POST
Dónde viajan los datos	En la URL como <i>query string</i> (?clave=valor&...)	En el cuerpo (<i>body</i>) del mensaje
Visibilidad	Los parámetros son visibles en la barra de direcciones, historial y logs	Los datos no aparecen en la URL
Request line	GET /ruta?param1=valor1 HTTP/1.1	POST /ruta HTTP/1.1
Headers relevantes	Host, User-Agent, Accept	Host, User-Agent, Content-Type, Content-Length
Cachable / Bookmarkable	✅ Sí, se puede guardar y compartir	❌ No, no se puede reproducir desde la URL
Idempotencia	✅ Normalmente idempotente (refrescar no reenvía datos nuevos)	❌ No idempotente (navegador pide confirmación de reenvío)
Uso típico	Consultas, búsquedas, filtros, datos no sensibles	Formularios de login, alta de datos, transacciones
🚩 Resumen rápido para memorizar:		
• GET = rápido, visible, ideal para consultas.		
• POST = oculto en body, ideal para datos sensibles o modificaciones en el servidor.		

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.

Cabecera Set-Cookie

- Es enviada por el servidor en la respuesta HTTP.
- Sirve para crear o actualizar una cookie en el cliente.

```
HTTP/1.1 200 OK
Set-Cookie: sessionId=abc123; Expires=Wed, 17 Sep 2025 23:59:59 GMT; Path=/; Secure; HttpOnly
```

Cabecera Cookie

- Es enviada por el cliente en el request siguiente.
- Informa al servidor los valores de cookies que tiene almacenados para ese dominio.

```
GET /perfil HTTP/1.1
Host: www.ejemplo.com
Cookie: sessionId=abc123
```

Relación con el funcionamiento de HTTP

- HTTP es sin estado (stateless): cada request es independiente.
- Las cookies permiten mantener estado entre requests, por ejemplo:
 - Identificar una sesión de usuario.
 - Guardar preferencias o información de carrito de compras.
- Sin cookies (o mecanismos similares como tokens/sesiones en el servidor), el servidor no podría “recordar” quién es el cliente entre dos requests consecutivos.

Resumen para tu apunte:

- **Set-Cookie:** el servidor crea o actualiza una cookie en el cliente.
- **Cookie:** el cliente envía de vuelta las cookies al servidor en requests posteriores.
- **Función principal:** permiten mantener información de sesión y personalizar la experiencia en un protocolo HTTP que por naturaleza no conserva estado.

19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?

Característica	Protocolo basado en texto	Protocolo binario
Formato	Los mensajes se transmiten en texto legible (ASCII/UTF-8).	Los mensajes se transmiten en formato binario (no legible directamente).
Legibilidad	Fácil de leer y depurar con herramientas simples (telnet, nc, etc.).	Difícil de interpretar sin herramientas específicas.
Tamaño de mensajes	Más grande (por cabeceras y separadores en texto).	Más compacto y eficiente en ancho de banda.
Velocidad de procesamiento	Más lento (necesita parsear cadenas de texto).	Más rápido (estructuras fijas, fácil de deserializar).
Ejemplos	HTTP/1.0, HTTP/1.1, SMTP, FTP (control)	HTTP/2, HTTP/3, gRPC, QUIC

Tipo de protocolo de HTTP

- HTTP/1.0: protocolo basado en texto.
- HTTP/1.1: también protocolo basado en texto, pero con mejoras (conexiones persistentes, cabeceras adicionales, etc.).
- HTTP/2: protocolo binario, más eficiente y soporta multiplexación de streams, compresión de cabeceras y priorización.

Resumen:

- HTTP/1.0 y HTTP/1.1: texto → fáciles de leer.
- HTTP/2: binario → más eficiente pero no legible directamente.

20. Responder las siguientes preguntas:

a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0?
 ¿Qué sucede en HTTP/2? (Ayuda: <https://undertow.io/blog/2015/04/27/An-in-depth-overview-of-HTTP2.html> para HTTP/2)?

Versión	Uso de la cabecera Host	Comentarios
HTTP/1.0	No definida en la especificación original. Algunos servidores la implementaron de forma opcional.	Sin Host, el servidor no puede diferenciar sitios en la misma IP → problema para hosting virtual.
HTTP/1.1	Obligatoria. Informa el nombre del host y puerto solicitado.	Permite servir múltiples sitios desde la misma IP (virtual hosting). Ejemplo: Host: <code>www.ejemplo.com</code>
HTTP/2	Se reemplaza por la pseudo-cabecera <code>:authority</code> , que cumple la misma función.	Los clientes suelen enviar también Host por compatibilidad, pero la especificación usa <code>:authority</code> como estándar.

HTTP1	HTTP2
<pre>GET /index.html HTTP/1.1 Host: www.ejemplo.com</pre>	<pre>:method: GET :scheme: https :authority: www.ejemplo.com :path: /index.html</pre>

b. En HTTP/1.1, ¿es correcto el siguiente requerimiento?

```
GET /index.php HTTP/1.1
User-Agent: curl/7.54.0
```

No, el requerimiento no es correcto en HTTP/1.1 porque falta la cabecera **Host**, que es obligatoria en esta versión.

Un servidor debería responder con **400 Bad Request** si no se incluye.

Forma correcta

```
GET /index.php HTTP/1.1
Host: www.ejemplo.com
User-Agent: curl/7.54.0
```

c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?

```
GET /index.php HTTP/1.1
Host: www.info.unlp.edu.ar
```

Se expresa en HTTP/2 con pseudo-cabeceras:

```
:method: GET
:scheme: https
:authority: www.info.unlp.edu.ar
:path: /index.php
```

Notas: en HTTP/2 las pseudo-cabeceras sustituyen a **Host** (`:authority`), y todo viaja en un formato binario con nombres de cabecera en minúsculas.